

CppHDL Best Practices

CppHDL is a C++ RTL framework for writing synthesizable hardware models while keeping the code executable as normal C++. The main advantages are:

- You write RTL in a familiar general-purpose language with templates, types, functions, and normal build tools.
- The same source can run as a fast native C++ model and can also be converted to SystemVerilog.
- Testbenches can be ordinary C++ programs, so random tests, reference models, file I/O, and debugging are simple.
- Large parameterized designs can use C++ templates instead of large preprocessor-heavy SystemVerilog code.
- CppHDL encourages small modules, explicit registers, and clear combinational/sequential separation.

Mapping of SystemVerilog Expressions to C++

CppHDL code should be written as a direct C++ mapping of synthesizable SystemVerilog RTL. Continuous assignments and module port connections belong in port member initializers or the `_assign()` section. This section runs only once, before the work cycle starts, and binds assignments that are used later during simulation and SystemVerilog generation. The `_ASSIGNxxx()` macros belong only in these static connection contexts, never in work, strobe, or combinational methods.

SystemVerilog (inside the parent module):

```
1 wire [31:0] child__data_in;
2 assign child__data_in = data_reg;
3 Child child (.clk(clk), .reset(reset), .data_in(child__data_in));
```

CppHDL (matching parent members and binding):

```
1 Child child;
2 reg<logic<32>> data_reg;
3
4 void _assign()
5 {
6     child.data_in = _ASSIGN_REG(data_reg);
7     child._assign();
8 }
```

Use `_ASSIGN(expr)` for expressions. Use `_ASSIGN_REG(reg_or_signal)` for direct storage bindings such as registers, logic values, memories, or ports whose final object reference is enough. Use `_ASSIGN_COMB(comb_func())` when assigning the result of a CppHDL combinational function. Both reference-binding macros take the address of an lvalue: do not pass a temporary, cast result, or by-value function call. `_ASSIGN_COMB()` invokes the comb chain on the first port read in a new `_system_clock` epoch; later reads reuse the cached result reference. For loop-indexed assignments use `_ASSIGN_I`, `_ASSIGN_REG_I`, `_ASSIGN_COMB_I`, or the indexed forms such as `_ASSIGN_INDEXED((i,j,k), expr)` and `_ASSIGN_REG_INDEXED((i,j,k), object[i][j][k])`.

For the default positive-edge clock, sequential logic maps into `_work(bool reset)`. Negative-edge logic uses `_work_neg(bool reset)` with `_strobe_neg()`; multi-clock designs use named work/strobe pairs (see the CDC chapter in `spec.md`). `_work()` computes next register values. It may contain the logic that would be split across several `always_ff` blocks in SystemVerilog.

SystemVerilog:

```

1  always_ff @(posedge clk) begin
2      if (reset) count_reg <= '0;
3      else if (enable_in) count_reg <= count_reg + 1;
4  end
5
6  always_ff @(posedge clk) begin
7      if (reset) valid_reg <= 1'b0;
8      else valid_reg <= enable_in;
9  end

```

CppHDL:

```

1  reg<u<8>> count_reg;
2  reg<u1> valid_reg;
3
4  void _work(bool reset)
5  {
6      if (reset) {
7          count_reg._next = 0;
8          valid_reg._next = 0;
9      }
10     else {
11         if (enable_in()) {
12             count_reg._next = count_reg + u<8>(1);
13         }
14         valid_reg._next = enable_in();
15     }
16 }

```

SystemVerilog `always_comb` blocks map to CppHDL combinational functions, usually named `*_comb_func()`. They calculate temporary combinational values from current inputs and

current register values. The usual style is to store the result in a member variable and return it by reference, as shown in the root examples.

SystemVerilog:

```
1 always_comb begin
2     hit = valid && tag == req_tag;
3     read_data = hit ? line[word] : '0;
4 end
```

CppHDL:

```
1 bool hit_comb;
2 bool& hit_comb_func()
3 {
4     hit_comb = valid_reg && tag_reg == req_tag_in();
5     return hit_comb;
6 }
7
8 u<32> read_data_comb;
9 u<32>& read_data_comb_func()
10 {
11     read_data_comb = hit_comb_func() ? line_reg[word_in()] : u<32>(0);
12     return read_data_comb;
13 }
14
15 _PORT(u<32>) read_data_out = _ASSIGN_COMB(read_data_comb_func());
```

CppHDL commits registers and memories in `_strobe()` (or the matching named/negative-edge strobe method). The testbench calls the top-level strobe after work evaluation; each parent explicitly calls its children's strobe methods. Register `.strobe()` calls and memory `.apply()` calls belong only in strobe methods, not in `_assign()`, `work`, or `comb` functions.

```
1 void _strobe()
2 {
3     count_reg.strobe();
4     valid_reg.strobe();
5     data_mem.apply();
6 }
```

Think Like RTL

CppHDL is C++, but the model should still be written as RTL. Keep the same mental model as SystemVerilog:

- Ports describe module inputs and outputs.
- `reg<T>` is state.
- `_work(reset)` computes next state.
- `_strobe()` commits next state to current state.
- Combinational functions compute outputs from current inputs/state.

SystemVerilog:

```

1 always_ff @(posedge clk) begin
2     if (reset) begin
3         count <= '0;
4     end else if (enable) begin
5         count <= count + 1;
6     end
7 end
8
9 assign done = (count == LIMIT);

```

CppHDL:

```

1 reg<u<8>> count;
2 _PORT(bool) enable_in;
3 _PORT(bool) done_out = _ASSIGN(count == LIMIT);
4
5 void _work(bool reset)
6 {
7     if (reset) {
8         count._next = 0;
9     }
10    else if (enable_in()) {
11        count._next = count + u<8>(1);
12    }
13 }
14
15 void _strobe()
16 {
17     count.strobe();
18 }

```

Ports

Use `_PORT(type)` for module and interface ports. A port is called like a function to read its value:

```

1 _PORT(bool) valid_in;

```

```

2  _PORT(logic<32>) data_in;
3
4  if (valid_in()) {
5      word._next = data_in();
6  }

```

Output ports are usually initialized with `_ASSIGN_REG(...)`, `_ASSIGN_COMB(...)`, or `_ASSIGN(...)`:

```

1  reg<u1> ready_reg;
2  _PORT(bool) ready_out = _ASSIGN_REG(ready_reg);
3  _PORT(bool) empty_out = _ASSIGN(count == 0);

```

`_ASSIGN_REG`, `_ASSIGN`, and Indexed Expressions

`_ASSIGN_REG(x)` connects a port to a persistent variable. It is the best choice for registers and member variables.

```

1  reg<logic<32>> data_reg;
2  _PORT(logic<32>) data_out = _ASSIGN_REG(data_reg);

```

`_ASSIGN(expr)` connects a port to an expression. Use it for simple computed outputs:

```

1  _PORT(bool) fire_out = _ASSIGN(valid_in() && ready_in());

```

`_ASSIGN_I(expr)` and `_ASSIGN_REG_I(expr)` are used in loops where the loop index must be captured for each generated connection:

```

1  void _assign()
2  {
3      uint32_t i;
4      for (i = 0; i < N; ++i) {
5          lanes[i].data_in = _ASSIGN_I(data_reg[i]);
6          lanes[i]._assign();
7      }
8  }

```

There are also `_J` and `_IJ` forms for nested loops.

Combinational Logic

For simple outputs, `_ASSIGN(...)` is enough. For larger combinational logic, write a member function that stores the result in a member variable and returns it by reference.

```
1 logic<32> result_comb;
2
3 logic<32>& result_comb_func()
4 {
5     result_comb = a_in() + b_in();
6     if (sub_in()) {
7         result_comb = a_in() - b_in();
8     }
9     return result_comb;
10 }
11
12 _PORT(logic<32>) result_out = _ASSIGN_COMB(result_comb_func());
```

Keep combinational functions side-effect-free except for assigning their own result variable. Call dependencies through their `*_comb_func()` methods; do not read another comb's result storage directly or rely on a separate preparation call.

Ordinary comb methods execute on every direct call. Use `_LAZY_COMB` to cache a result per `_system_clock` epoch:

```
1 _LAZY_COMB(sum_comb, logic<32>)
2     sum_comb = a_in() + b_in();
3     return sum_comb;
4 }
```

This declares both `sum_comb` storage and `sum_comb_func()`. Assign the complete result on every evaluation path to avoid stale values and inferred latches.

Sequential Logic

Write next-state logic in `_work(reset)` and commit it in `_strobe()`.

```
1 void _work(bool reset)
2 {
3     if (reset) {
4         valid_reg._next = 0;
5         data_reg._next = 0;
6     }
7     else {
8         valid_reg._next = next_valid;
```

```

9         data_reg._next = next_data;
10     }
11 }
12
13 void _strobe()
14 {
15     valid_reg.strobe();
16     data_reg.strobe();
17 }

```

Call nested module hooks explicitly:

```

1 void _work(bool reset)
2 {
3     child._work(reset);
4     if (reset) {
5         state._next = 0;
6     }
7     else {
8         state._next = child.data_out();
9     }
10 }
11
12 void _strobe()
13 {
14     child._strobe();
15     state.strobe();
16 }

```

CppHDL and SystemVerilog Look Similar

A valid-ready transfer in SystemVerilog:

```

1 assign out_valid = valid_reg;
2 assign out_data  = data_reg;
3
4 always_ff @(posedge clk) begin
5     if (reset) begin
6         valid_reg <= 1'b0;
7     end else if (!valid_reg || out_ready) begin
8         valid_reg <= have_data;
9         data_reg  <= next_data;
10    end
11 end

```

The same idea in CppHDL:

```

1  _PORT(bool) valid_out = _ASSIGN_REG(valid_reg);
2  _PORT(logic<32>) data_out = _ASSIGN_REG(data_reg);
3  _PORT(bool) ready_in;
4
5  void _work(bool reset)
6  {
7      if (reset) {
8          valid_reg._next = 0;
9      }
10     else if (!valid_reg || ready_in()) {
11         valid_reg._next = have_data;
12         data_reg._next = next_data;
13     }
14 }

```

Use C++ Templates for Parameters

Prefer templates for structural parameters such as data width, address width, number of ports, or FIFO depth.

```

1  template<size_t DATA_WIDTH, size_t DEPTH>
2  class Fifo : public Module
3  {
4      static_assert(DEPTH > 0, "DEPTH must be positive");
5  public:
6      _PORT(logic<DATA_WIDTH>) write_data_in;
7  private:
8      reg<u<(DEPTH <= 1 ? 1 : clog2(DEPTH))>> write_ptr;
9      // Work and strobe methods omitted from this declaration excerpt.
10 };

```

This keeps parameterized RTL type-safe and avoids many preprocessor tricks.

Interfaces

Use `Interface` for bundles containing signals of different directions. You do not need separate driver and responder interface types.

```

1  template<size_t DATAWIDTH>
2  struct ValidReadyIf : public Interface
3  {
4      _PORT(bool) valid_in;
5      _PORT(bool) ready_out;
6      _PORT(logic<DATAWIDTH>) data_in;
7  };

```

A source and sink can both use `ValidReadyIf`. A parent module or test wrapper can connect them with `assignIf()`:

```
1 VRDriver<32> driver;
2 VRResponder<32> responder;
3
4 void _assign()
5 {
6     assignIf(driver, responder, driver.source_out, responder.sink_in);
7 }
```

`assignIf()` performs the bidirectional binding order needed when one side drives `valid/data` and the other drives `ready`; it invokes the endpoint modules' `_assign()` methods during setup. Connect an interface as a bundle instead of manually copying its individual ports. Endpoint modules still bind the signals they themselves drive.

Make connections in the immediate common parent. Do not reach through a child into a grandchild or assign another module's internal state. Forward an interface through a proxy with `assignIf(*this, child, proxy_in, child.sink_in)`; see `tests/interface/AssignIfHierarchyProxy.cpp`.

Arrays, Logic, and Bit Ranges

Use `logic<N>` for arbitrary-width bit vectors and `u<N>` for unsigned integers up to 64 bits. Use `.bits(hi, lo)` and `operator[]` for bit slicing.

```
1 logic<64> word;
2
3 word.bits(31, 0) = low_word;
4 word.bits(63, 32) = high_word;
5 word[0] = parity_bit;
```

Use `array<COUNT, TYPE, PACKED = false>` (count first). The default uses unpacked C++ element storage; `true` uses packed storage. Use `memory<TYPE, ROW_SIZE, DEPTH>` for deferred-write memories, where `ROW_SIZE` counts elements per row, not bits.

```
1 array<16, u8> bytes;
2 array<16, u8, true> packed_bytes;
3 array2D<4, 8, u16> matrix;
4 memory<u8, 4, 256> words; // 256 rows of four bytes.
```

See `spec.md` for the distinction between C++ storage and generated SV packing. Synthesizable indexes and loop variables should use `uint32_t` or a narrower type, not 64-bit `size_t`.

Keep Synthesizable Code Simple

CppHDL can run any C++ in native simulation, but converted RTL should use a synthesizable subset:

- Variables can be declared inside methods/functions, but put all declarations at the very beginning of the method/function.
- Avoid dynamic allocation in RTL models.
- Avoid STL containers in synthesizable state.
- Prefer fixed-size CppHDL types: `u<>`, `logic<>`, `array<>`, `memory<>`, and structs.
- Put file I/O, randomization, and reference models in inline tests, not in RTL modules.
- Do not assume default-constructed signals are zero. Assign local values before reading them and explicitly reset required register state.
- Prefer `if (reset) { ... } else { ... }` to early returns from work methods, and always propagate reset to child work methods.

Inline Tests

Examples and tests usually keep the first test in the same `.cpp` file as the RTL model. This is the CppHDL inline test practice: the file contains synthesizable modules first, then a `// CppHDL INLINE TEST` section with ordinary C++ test code.

An inline test normally does four things:

- Instantiates the CppHDL model.
- Drives input ports with C++ values.
- Runs the RTL cycle hooks: `_assign()`, `_work(reset)`, and `_strobe()`.
- Checks outputs and internal behavior against expected values or a C++ reference model.

Complete single-clock native testbench shape (assuming `MyModule` has `enable_in` and `done_out` ports):

```
1 // CppHDL INLINE TEST //////////////////////////////////////
2 #if !defined(SYNTHESIS)
3 #include <cassert>
4
5 long _system_clock = -1; // Define once per native simulation executable.
6
7 class MyModuleTest : public Module
8 {
9 public:
10     MyModule dut;
11     bool enable = false;
12
```

```

13  void _assign()
14  {
15      dut.enable_in = _ASSIGN(enable);
16      dut._assign();
17  }
18
19  void tick(bool reset)
20  {
21      ++_system_clock;
22      dut._work(reset);
23      dut._strobe();
24      ++_system_clock;
25  }
26 };
27
28 int main()
29 {
30     MyModuleTest test;
31     test._assign(); // Once, outside the cycle loop.
32     test.tick(true);
33     assert(!test.dut.done_out());
34     test.enable = true;
35     test.tick(false);
36 }
37 #endif

```

Keep randomization, file I/O, scoreboards, and reference models in the inline test, not in synthesizable RTL classes. `SYNTHESIS` guards hide the testbench from conversion. Testbenches using `std::print` require a standard library with print support; C++17 consumers can use `printf` or streams instead. CppHDL does not provide a replacement `std::print`.

`_system_clock`

Native ports and lazy combs use the global `long _system_clock` declared by `cpphdl_port.h`. Define it once in the testbench, not once per module. The name is `_system_clock`, not `sys_clock`.

This is a cache-invalidation epoch, not a hardware clock or elapsed-time counter. Starting from `-1`, increment it before the first evaluation. Increment it after input changes and after committing state before reading outputs again. The tick above starts a fresh epoch before work and another after strobe; use a separate counter for simulated clock cycles.

For simultaneous clock edges, evaluate all active work methods against pre-edge state before calling any strobe methods. Do not rebind ports or call `_assign()` in the cycle loop. See the CDC and asynchronous-reset chapters in `spec.md` for named-clock scheduling.

Build and Run Tests

The `examples/` and `tests/` folders are built by CMake. Most `.cpp` files become executable targets with post-build SystemVerilog generation; optimizer and other special regressions use dedicated CMake rules.

Configure once:

```
1 cmake -S . -B build
```

Build one test:

```
1 cmake --build build --target interface_ValidReady
```

Run the native CppHDL test:

```
1 (cd build/tests && ./interface_ValidReady --noveril)
```

Run the full test, including the Verilator part when the inline test supports it:

```
1 (cd build/tests && ./interface_ValidReady)
```

Run from `build/tests` because the inline harness locates `generated/` relative to its working directory. Alternatively, use `ctest --test-dir build -R '^interface_ValidReady$' --output-on-failure` for the registered test.

For a new test, place it under `tests/<group>/<Name>.cpp`. The target name is derived from the relative path, so `tests/interface/ValidReady.cpp` becomes `interface_ValidReady`.

Conclusion

CppHDL runs RTL directly as native C++. No translator is used for native simulation, so compile/debug/test cycles are fast and use ordinary C++ tooling. Performance depends on the design, compiler, optimizer mode, and testbench; benchmark the actual workload. The same RTL source can still be converted to SystemVerilog for verification and implementation.